

## LECTURE 9

# Aggregation and Grouping

Summarizing many tuples into meaningful answers

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

## What This Lecture Covers

- 1 Why relational algebra's five basic operators cannot answer summary questions
- 2 The aggregate functions: COUNT, SUM, AVG, MIN, MAX
- 3 The aggregate operator  $\mathcal{F}$  (script F)
- 4 The grouping operator  $\mathcal{G}$  (script G) and grouping attributes
- 5 Combining grouping with aggregation to answer per-group questions
- 6 Worked examples and common pitfalls

## A New Kind of Question

- $\sigma$ ,  $\pi$ , and  $\bowtie$  always return a relation of the same general shape — still rows and columns, one tuple per matching input row.
- “What is the total salary bill?” is not about which tuples to keep — it needs every tuple reduced to a single number.
- No  $\sigma$ ,  $\pi$ , or  $\bowtie$  expression can produce a value computed across many tuples — relational algebra needs a new operator family.

*Sum of salary = 30000 + 12000 + 18000 + 9000 + 22000 = 91000*

**Staff (staffNo, name, position, salary, branchNo)**

| staffNo | name       | position   | salary | branchNo |
|---------|------------|------------|--------|----------|
| SL21    | John White | Manager    | 30000  | B005     |
| SG37    | Ann Beech  | Assistant  | 12000  | B003     |
| SG14    | David Ford | Supervisor | 18000  | B003     |
| SA9     | Mary Howe  | Assistant  | 9000   | B007     |
| SL41    | Julie Lee  | Manager    | 22000  | B005     |

## Five Standard Aggregate Functions

| Function | Meaning                       | Applies to          |
|----------|-------------------------------|---------------------|
| COUNT    | Number of values (or tuples)  | Any type            |
| SUM      | Total of all values           | Numeric only        |
| AVG      | Arithmetic mean of all values | Numeric only        |
| MIN      | Smallest value                | Numeric / orderable |
| MAX      | Largest value                 | Numeric / orderable |

## Applying the Functions to Staff.salary

| Function      | Result |
|---------------|--------|
| COUNT(salary) | 5      |
| SUM(salary)   | 91000  |
| AVG(salary)   | 18200  |
| MIN(salary)   | 9000   |
| MAX(salary)   | 30000  |

**NOTE**

COUNT is the only aggregate that works on any column, including non-numeric ones, and COUNT(\*) counts tuples directly, ignoring columns entirely.

## The Aggregate Operator, $\mathcal{F}$

$\mathcal{F}$  <function-list> (R)

- $\mathcal{F}$  applies one or more aggregate functions across an entire relation.
- With no grouping attribute specified, the whole relation is treated as one group.
- The result is always a relation with exactly one tuple — no matter how many input rows.

```
 $\mathcal{F}$  SUM(salary), AVG(salary)  
      (Staff)
```

### Result

| SUM(salary) | AVG(salary) |
|-------------|-------------|
| 91000       | 18200       |

#### WARNING

Aggregation is not filtering —  $\mathcal{F}$  AVG(salary)(Staff) does NOT list each staff member alongside the average. It discards every individual row and returns only the single computed number.

## Grouping Splits a Relation into Buckets

- Grouping partitions a relation into groups sharing the same value of one or more grouping attributes.
- Grouping alone adds no new information — it's purely organizational, until an aggregate is applied to each group.
- Every distinct value of the grouping attribute gets exactly one group — groups are never declared in advance.



*Grouping Staff by branchNo, before any aggregate is applied*

## A Second Grouping: by Position

- The same five Staff rows, partitioned by a completely different attribute: position instead of branchNo.
- John White and Julie Lee happen to both be Managers, which is why this grouping looks similar to the branchNo one — that overlap is a coincidence of this data, not a rule.
- Ann Beech and Mary Howe share the Assistant position even though they work at different branches, B003 and B007.



*Grouping Staff by position instead of branchNo — same five rows, a different partition*

## The Grouping Operator, $\mathcal{G}$

```
<grouping-attributes>  $\mathcal{G}$   $\mathcal{F}$  <function-list> (R)
```

- Read left to right: partition R into groups sharing the same value of grouping-attributes, then compute the function-list separately within each group.
- The result has one tuple per group, rather than one tuple for the whole relation.

### TIP

Grouping by the wrong attribute is a silent logic error, not a syntax error — the query still runs and still returns a plausible-looking table. State the question in plain English first.

## WORKED EXAMPLE 1

# Total Salary Per Branch

```
branchNo | SUM(salary) | (Staff)
```

- B005: John White (30000) + Julie Lee (22000) = 52000
- B003: Ann Beech (12000) + David Ford (18000) = 30000
- B007: Mary Howe alone = 9000

## Result

| branchNo | SUM(salary) |
|----------|-------------|
| B005     | 52000       |
| B003     | 30000       |
| B007     | 9000        |

## Count of Staff Per Position

```
position G F COUNT(staffNo) (Staff)
```

- Managers: John White, Julie Lee — group size 2
- Assistants: Ann Beech, Mary Howe — group size 2
- Supervisor: David Ford — group size 1
- Changing the grouping attribute changes the entire shape of the result.

### Result

| position   | COUNT(staffNo) |
|------------|----------------|
| Manager    | 2              |
| Assistant  | 2              |
| Supervisor | 1              |

# Pause and Compute It Yourself

```
position G F AVG(salary) (Staff)
```

- Use the position groups from two slides back: Manager, Assistant, Supervisor.
- Average = sum of the salaries in a group, divided by how many staff are in that group.
- Work out all three averages by hand, then compare them with the next slide.

Your Task

| position   | AVG(salary) |
|------------|-------------|
| Manager    | ?           |
| Assistant  | ?           |
| Supervisor | ?           |

## Answer: Average Salary Per Position

```
position G F AVG(salary) (Staff)
```

- Manager:  $(30000 + 22000) / 2 = 26000$
- Assistant:  $(12000 + 9000) / 2 = 10500$
- Supervisor: 18000 alone, so the average is just 18000
- Same grouping attribute as the practice question — only the aggregate function changed, from COUNT to AVG.

### Result

| position   | AVG(salary) |
|------------|-------------|
| Manager    | 26000       |
| Assistant  | 10500       |
| Supervisor | 18000       |

# Multiple Aggregates in One Grouped Query

```
branchNo  G F COUNT(staffNo), AVG(salary), MAX(salary) (Staff)
```

Result

| branchNo | COUNT(staffNo) | AVG(salary) | MAX(salary) |
|----------|----------------|-------------|-------------|
| B005     | 2              | 26000       | 30000       |
| B003     | 2              | 15000       | 18000       |
| B007     | 1              | 9000        | 9000        |

B005's average is  $(30000 + 22000)/2 = 26000$ , maximum is John White's 30000. B003's average is  $(12000 + 18000)/2 = 15000$ . B007 has only one row, so count, average, and maximum all equal Mary Howe's single salary.

## SECOND EXAMPLE

### PropertyForRent: Setting Up the Query

PropertyForRent (propertyNo, street, city, type, rooms, rent, branchNo)

| propertyNo | city      | type  | rent  | branchNo |
|------------|-----------|-------|-------|----------|
| PA14       | Islamabad | House | 42000 | B007     |
| PL94       | Karachi   | Flat  | 28000 | B005     |
| PG4        | Lahore    | Flat  | 21000 | B003     |
| PG36       | Lahore    | Flat  | 26000 | B003     |
| PG21       | Lahore    | House | 39000 | B003     |

```
type G F
COUNT (propertyNo) ,
AVG (rent)
(PropertyForRent)
```

## Average Rent Per Property Type — Result

```
type G F
COUNT (propertyNo) ,
AVG (rent)
(PropertyForRent)
```

### Result

| type  | COUNT | AVG(rent) |
|-------|-------|-----------|
| House | 2     | 40500     |
| Flat  | 3     | 25000     |

*Flat:  $28000 + 21000 + 26000 = 75000 \div 3 = 25000$ . House:  $42000 + 39000 = 81000 \div 2 = 40500$ .*

## Filter Before You Aggregate

*“Find the total rent collected on properties managed by branch B003”*

**$\sigma$  branchNo = 'B003'**  
Filter first: keep PG4, PG36, PG21



**$\mathcal{F}$  SUM(rent)**  
21000+26000+39000 = 86000

## Pitfall: Grouping by the Wrong Attribute

### WARNING

branchNo  $\mathcal{G} \mathcal{F}$  SUM(rent) (PropertyForRent) gives B003 = 86000, B005 = 28000, B007 = 42000: one row per branch. type  $\mathcal{G} \mathcal{F}$  SUM(rent) (PropertyForRent) gives House = 81000, Flat = 75000: one row per property type, completely different numbers. Both run without error and both look equally plausible — only the English question you started from tells you which one is correct. Always state the question in plain English first, and let that sentence choose the grouping attribute.

## $\mathcal{F}$ and $\mathcal{G}$ — Side by Side

```
<grouping-attributes>  $\mathcal{G}$   $\mathcal{F}$  <function-list> (R)
```

- $\mathcal{F}$  function-list (R) alone collapses the whole relation into exactly one result tuple — no grouping attribute means the entire relation is treated as a single group.
- grouping-attrs  $\mathcal{G}$   $\mathcal{F}$  function-list (R) applies the same functions separately to each group, producing one result tuple per group instead of one for everything.

### TIP

If the question reads “...for the whole table,” reach for  $\mathcal{F}$  alone. If it reads “...for each X,” you need  $X \mathcal{G} \mathcal{F}$  — the English phrasing tells you which one to use.

## KEY TAKEAWAYS

# Aggregation and Grouping

- 1 COUNT, SUM, AVG, MIN, MAX reduce many values from one column into a single summary value.
- 2 The aggregate operator  $\mathcal{F}$  collapses an entire relation into one result tuple when no grouping attribute is given.
- 3 Grouping ( $\mathcal{G}$ ) partitions a relation into buckets sharing a grouping-attribute value — by itself it computes nothing.
- 4 grouping-attrs  $\mathcal{G}$   $\mathcal{F}$  function-list (R) applies aggregates separately per group, one result tuple per group.
- 5 Filter with  $\sigma$  before grouping; choosing the wrong grouping attribute is a silent logic error, not a syntax error.